Technische Universität Berlin
Faculty IV: Electrical Engineering and Computer Science
Freie Universität Berlin
Department of Mathematics and Computer Science
Algorithmic Bioinformatics

welche uni angeben, welche reihenfolge..?

Automatic Optimization of Level Parameters for RSHash

(RSHash is a minimizer based fast, compressed hash table for storing DNA k-mers. It was recently introduced by the Reinert Group.)

Bjarne David Jehn

Thesis submitted in fulfillment of the requirements for the degree "Bachelor of Science"(B. Sc.) in the field of Computer Science

First Reviewer:	Prof. Dr. Knut Reinert
Second Reviewer:	Prof. Dr. Stephan Kreutzer
Supervisor:	Jonas Schulte-Mattler

Declaration of Authorship

I hereby declare that the thesis submitted is my own, unaided work, completed without any external help. Only the sources and resources listed were used. All passages taken from the sources and aids used, either unchanged or paraphrased, have been marked as such.

Where generative AI tools were used, I have indicated the product name, manufacturer, the software version used, as well as the respective purpose (e.g. checking and improving language in the texts, systematic research). I am fully responsible for the selection, adoption, and all results of the AI-generated output I use.

I have taken note of the Principles for Ensuring Good Research Practice at TU Berlin dated 15 February 2023. https://www.static.tu.berlin/fileadmin/www/10002457/K3-AMBl/Amtsblatt_2023/Amtliches_Mitteilungsblatt_Nr._16_vom_30.05.2023.pdf

I further declare that I have not submitted the thesis in the same or similar form to any other examination authority.

Berlin, _____ _____

Date Signature

Abstract

in English

RSHash is a static, compressed data structure for k-mers that answers lookup queries using a leveled minimizer scheme with thresholds to cut off frequent minimizers. Building it requires specifying a minimizer lengths and a threshold for every level used. Both its space consumption and the lookup time depend on these parameters because they affect the distribution of minimizers. We

Zusammenfassung

auf Deutsch

Contents

1 Introduction **5**

2 Previous Work **7**

3 Notation **10**

4 Sequence Statistics **11**
4.1 Minimizer Distribution 11
4.2 Average Frequencies 13
4.3 Numbers of Distinct *Infrequent* Minimizers 13
4.4 Times for Operations on Data Structures 13

5 MINLP Modeling **15**
5.1 Model Input 15
5.2 Model for One Level 15
5.2.1 Lookup Time 16
5.2.2 Space Consumption 18
5.2.3 Optimization Goal 19
5.3 Model for Several Levels 22
5.3.1 Lookup Time 23
5.3.2 Space Consumption 24

6 Experiments **26**
6.1 Datasets 26
6.2 Considered Parameters 26
6.3 Computing the Model Input 27
6.4 Model Accuracy 29
6.4.1 Lookup Time 29
6.4.2 Space Consumption 29
6.5 Limitations 30

7 Outlook **31**

8 Conclusion **32**

9 stuff **33**

Bibliography **35**

1 Introduction

Genetic strands are chains of the nucleobases adenine (A), cytosine (C), guanine (G), thymine (T) and uracil (U). Adenine, cytosine and guanine are present in both DNA and RNA whilst thymine only appears in DNA and uracil is specific to RNA. A genetic sequence can thus be represented as a string over an alphabet of length four meaning a compact representation of two bits needed to store each base. Despite this, genetic databases are often very large as sequences usually consist of several billion bases e.g. approximately $3.1 \cdot 10^9$ bases for the human genomes sequences.
The rapid growth of available genetic data due to better and faster sequencing techniques has led to a need for efficient algorithms for processing genetic sequences.

K-mers have emerged as an essential component of many approaches to encounter this issue. The k-mers of a sequence are all its substrings of length k, thus a sequence S with $|S| \geq k$ contains exactly $|S| - k + 1$ k-mers; for example, the sequence $CGACAT$ consists of the 3-mers CGA, GAC, ACA, CAT.
Uses for k-mers include k-mer counting, frequency analysis, sequence indexing, genome assembly, sequence comparison, taxonomic classification, phylogeny reconstruction, protein sequence searching and alignment as well the construction of k-mer databases [Moe+24].

Sequence comparison is used to compare a sequence against large genomic reference datasets; for example, to classify to what species a collected DNA sample belongs or to analyze whether a patients DNA contains indicators linked to certain diseases.
One approach to this is to check whether each k-mer in the sequence of interest is present in the reference dataset, also referred to as a lookup of the queried k-mers. On the other hand, a so called locate query answers not only whether queried k-mers are present but also returns all positions of the queried k-mers in the sequences. These positions are referred to as their offsets.

A trade-off between space consumption and lookup time is an inherent problem for data structures designed to answer lookup or locate queries. This can be demonstrated by considering the two extremes for data structures enabling a lookup or locate of a queried k-mer q in a sequence S with $|S| \gg k$ that doesn't contain any duplicate k-mers[1]:
The most space efficient representation of all k-mers in S is the sequence itself needing approximately two bits per k-mer (bpk). A lookup/locate query of a q would require scanning through the whole sequence, comparing every k-mer with q and returning

[1] For sequences containing duplicate k-mers, the lookup case does not change much. However, in the locate case it would then be required to find all offsets of q. For the compact example this would require to always scan through the whole sequence. For the fast example this would mean that the hash table entry for each k-mer would store a list of all its offsets.

whether q appears in the sequence (and its offset for the locate case). Both queries would terminate upon either finding a match or reaching the sequences end, requiring $O(|S|)$ time.
A very fast lookup/locate on the other hand could be achieved by storing the appearance of every k-mer in S in a hash set (locate: hash table) enabling both queries in $O(1)$ time. For this approach every k-mer would have to be stored in the hash set/table leading to a space consumption of $2k$ bits per k-mer in the lookup case and $2k + log_2(|S|)$ bits per k-mer to enable locate queries which require the offsets to be stored as well.
Since both a lookup time in $O(|S|)$ as well as $2k$ bits per k-mer are not viable for large genomes, there has been much work on data structures enabling queries in much less than $O(|S|)$ whilst also maintaining a certain compression.

Using so called minimizers is a common approach to achieve this. A minimizer of length m is the smallest m-mer of a k-mer regarding a total order over all strings of length m. Using the lexicographical order for example to determine the minimizer of length 2 in the 6-mer $CGACAT$ results in AC because regarding that order, this is the smallest 2-mer in $CGACAT$.
Minimizers have several properties which render them highly interesting for this domain. Firstly they are considered to be sparse because consecutive k-mers are very likely to share the same minimizer. Zheng, Kingsford, and Marçais [ZKM20] show that in a random string S with $|S| \gg k > m$ each minimizer of length m is expected to cover an average of $(k - m + 2)/2$ k-mers. These consecutive k-mers sharing the same minimizer are also referred to as super-k-mers. Even in a sequence only containing unique k-mers, several super-k-mers can have the same minimizer as $m < k$.
A second interesting property is the skewness of the distribution of minimizers in regards to their *frequencies* which is the number of super-k-mers which have a specific minimizer. In a given sequence only a few minimizers have high frequencies whilst most have small frequencies or do not appear at all. Some intuition for this property can be gained by considering an m-mer consisting only of As versus one of Ts. Using the lexicographical order the first will be the minimizer of every super-k-mer in which it appears whilst the second would only become the minimizer of a super-k-mer containing only Ts.
Both the sparsity and skewness of minimizers have been observed by several previous works [Chi+14; Jai+20; Pib22; SMR26].

2 Previous Work

The SSHash algorithm (Sparse and Skew hashing of K-mers) introduced by Pibiri [Pib22] exploits the two properties of minimizers being sparsely and skewly distributed by computing all minimizers appearing in a sequence and then use a minimal perfect hash function to store every super-k-mers offsets in an array *Offsets* in the order induced by their minimizers hashes. In a second array *Sizes* the nth element stores the starting index in *Offsets* for the offsets belonging to the nth minimizer. The sparsity of minimizers mainly effects the data structures space consumption: By indexing super-k-mers, they can compress the data structures size by a factor of $(k-m+2)/2$ compared to indexing individual k-mers. The skewness of minimizers on the other hand is of importance to the locate time: Querying a k-mer q requires computing its minimizer and then searching for it in every super-k-mer with the same minimizer. For a sufficiently large minimizer length, minimizers of high frequency become increasingly unlikely.
The both space consumption and locate time of SSHash are highly dependent of the minimizer length m: A smaller m means a higher compression $(k-m+2)/2$. The lookup/locate time on the other hand is largest for small m because those result in minimizers of higher *frequencies*. This demonstrates the challenge that comes with choosing an optimal minimizer length.

The RSHash algorithm (Rank-Select Hash) which was recently introduced by Schulte-Mattler and Reinert [SMR26] is a minimizer based fast, compressed hash set for storing DNA k-mers that answers lookup and can be extended for supporting locate. It follows a minimizer based approach like SSHash; however, instead of using minimal perfect hashing, RSHash relies on bitvectors supporting rank and select queries. Its main difference to previous methods is the introduction of a layered multi-minimizer scheme aiming to reduce the cost of high *frequencies* whilst at also maintaining large compression.
The data structure works with one, two or three so called levels and a hash set. In order to build RSHash to answer lookup (or locate) queries one has to specify the number of levels as well as a pair of level parameters for each level l: the minimizer length m^l and a threshold t^l.

Building RSHash: When using only one level, that level is build by first computing each k-mers minimizer of length m^1 using a hash function h_1 and counting the number of times each minimizer occurs in the sequences. We call minimizers that appear at least once but no more than the threshold t^1 to be *infrequent* on the level.
A bitvector R^1 of lengths 4^{m^1} is used to mark the appearance of each distinct *infrequent* minimizers. For this, the minimizers are interpreted as binary numbers of $2m^1$ bits and serve directly as indices to the vector. If for example the minimizers length were chosen to be $m^1 = 2$ and AA was an *infrequent* minimizer in the sequences, then $R^1[AA] = R^1[0]$

Frage: wie soll ich die hash funk-tion beschreiben

would be set to one.
The offsets each *infrequent* minimizers occurrences in the sequences are stored in the offset vector O^1 in the order induced by R^1. Let f for example be the frequency of the smallest *infrequent* minimizer, that is the number of occurrences of the smallest minimizer for which the bit in R^1 is set. Then $O^1[0 : n)$ contains the offsets of that minimizer whilst the next offsets $O^1[n : ...)$ belong to the second smallest minimizer and so forth.
The boundaries of each minimizers offsets in O^1 are marked in the bitvector S^1 of length $|O^1| + 1$. Following up on the example above, the first bit $S^1[0]$ would be set to one, indicating the start of the offsets belonging to the smallest minimizer, this would be followed by zeros until the nth bit $S^1[n] = 1$ indicating the start of the offsets belonging to the second smallest minimizer and so forth. The last bit in S is also set to one marking the end of the largest minimizers offsets.

If a second level is specified, only those super-k-mers with *frequent* minimizers on the first level are handled on the second level which is build almost identically to the first level, creating the vectors R^2, O^2 and S^2 but using the hash function h_2 for the computation of minimizers. The use of a different hash function for the computation leads to a skewed distribution of minimizers, in which most k-mers having *frequent* minimizers on the first level, have *infrequent* ones on the second. This holds even when the specified minimizer length m^2 and threshold t^2 are equal to those on the first.
A third level would be build analogously, if specified.

Only the appearance of k-mers that have *frequent* minimizers on all specified levels are then directly stored in a fast hash set sacrificing some space in return for a fast lookup which is viable as those k-mers are only a small fraction of the whole input sequence for a good choice of the level parameters.

However, the choice of "good" parameters for a given sequence remains an open problem. We already noted that using a small $m \ll k$ leads to a better compression but at the same time results in longer query times. This inverted effect of the minimizer length on the space consumption and query time is an inherent problem of minimizer based k-mer indexing methods.
Existing methods use a number of approaches to deal with this problem. For example, Pibiri [Pib22] chooses a relatively large minimizer length, accepting a smaller compression in order to maintain reasonable locate times.

The leveled approach introduced in the RSHash algorithm on the other hand aims to achieve high compression by using relatively short minimizers on the lower levels whilst at the same time mitigating the burden of very *frequent* minimizers by using thresholds to cut them off and guaranty fast queries on every level. The use of distinct hash function to compute minimizers on each level leads to different skewed distributions of the minimizers on each level. Because only a small percentage of the k-mers are frequent for each distribution, the number of k-mers reduces drastically with each level and only a small percentage of the sequences k-mers end up being stored in the spatially expensive hash set.

RSHash thus allow significantly faster queries opposed to SSHash whilst remaining similarly compact. However, this is highly dependent of both the chosen minimizer

lengths which influence the sparsity and skewness of minimizer distributions on each level and of the chosen thresholds, which affect both the lookup time on each level as well as the percentage of k-mers having to be stored in the hash set. For example, choosing very short minimizers and small thresholds on every level would result in a very large hash set as almost no minimizer would be *infrequent* .

Choosing "good" level parameters to build the data structure for a given set of sequences currently requires manual testing and extensive understanding of the RSHash algorithm. We attempt to model the space consumption and lookup time of RSHash depending on the level parameters aiming to expand the understanding about how those values are influenced by the parameters and to work towards automating the choice of optimal level parameters for a given genome and optimization goal.

3 Notation

A sequence is a string over the DNA alphabet $\Sigma = \{A, C, G, T\}$. Let $i, j \in \mathbb{N}$ with $i < j$ be zero based indices of a string S of length $|S| \geq j$. We use $S[i : j)$ to denote the substring from the ith to the $(j-1)$th character of S. The k-mers of a string S with $|S| \geq k$ are all its substrings $S[i : i + k) \in \Sigma^k$ of length k for $i \in \mathbb{N} \wedge i \leq |S| - k$.
Let $L \in \mathbb{N}^+$ be the number of levels specified. We refer to the set of levels $\mathcal{L} := \{l \in \mathbb{N}^+ \mid l \leq L\}$. We use $\mathcal{M} \subseteq [1, k) \cap \mathbb{N}$ and $\mathcal{T} \subset \mathbb{N}^+$ to denote the sets of minimizer lengths and thresholds considered. We denote the Cartesian nth power of a set X with X^n e.g. $X^3 = X \times X \times X$. A chosen minimizer length and threshold on a specified level $l \in \mathcal{L}$ are denoted m^l and t^l.
We use the tuple $p^l := (m^1, ..., m^l, t^1, ..., t^l) \in \mathcal{M}^l \times \mathcal{T}^l$ to denote the level parameters up to level l. For a specified tuple p^l of level parameters we use p^{l-1} to denote the level parameters $(m^1, ..., m^{l-1}, t^1, ..., t^{l-1})$ up to the previous level. Furthermore, we introduce the operator $p^l[m^i] = m^i$ to denote retrieving the minimizer length specified for level i from the level parameters and analogously $p^l[t^i] = t^i$.
We write $S_{p^{l-1}} := \{S_1^l, S_2^l, ..., S_n^l\}$ to denote the set of sequences on level l in dependence of the level parameters p^{l-1} on the previous levels i.e. the k-mers which have *infrequent* minimizers on all previous levels. The full set of input sequences when building RSHash do not depend on any parameters, are denoted with S_{p^0} and are sometimes referred to as the input "text".
Let $q \in \Sigma^k$ be a k-mer. The computation of its minimizer $min_{m^l}^{h_l}(q)$ on level l depends both on the specified minimizer length m^l as well as the hash function h_l on that level, which is not dependent of the level parameters but unique for every level. We denote the minimizers frequency with $freq(min_{m^l}^{h_l}(q))$ which tells the combined number of the minimizer occurrences in a given set of input sequences.
We call the minimizer of q *infrequent* on level l only if $1 \leq freq(min_{m^l}^{h_l}(q)) \leq t^l$ holds i.e. the minimizer occurs at least once but no more than the specified threshold on level l. Minimizers with frequencies larger than the threshold are referred to as being *frequent* .

4 Sequence Statistics

As outlined in the Chapter 1, the space consumption and query time of RSHash are highly dependent on the choice of level parameters as those influence the distributions of minimizers on all levels. We define aspects of the minimizer distribution which will be needed for modeling the space consumption and query time in Chapter 5. Specifically relevant for this is the behavior of minimizer distributions across several levels, which is unique to the approach proposed by Schulte-Mattler and Reinert [SMR26] and therefor not considered in previous works.

4.1 Minimizer Distribution

Recall that we use $p^l \in \mathcal{M}^l \times \mathcal{T}^l$ to describe the level parameters on all levels up to and including the level l and that S_{p^0} is the full set of input sequences on the first level, which does not depend on any parameters. Also recall that the set $S_{p^{l-1}}$ of input sequences on level l depends on the level parameters p^{l-1} on all previous level, as those affect, whether k-mers minimizer is *frequent* or *infrequent* .

Let $\mathcal{K}_{p^{l-1}}$ be the set of distinct k-mers in a set of sequences $S_{p^{l-1}}$. In other words, the set $\mathcal{K}_{p^{l-1}}$ contains every distinct substring of length k which appears in at least one of the sequences $S_i^l \in S_{p^{l-1}}$ which depend on the level parameters on previous levels:

$$\mathcal{K}_{p^{l-1}} := \{q \in \Sigma^k \mid \exists S' \in S_{p^{l-1}} \; \exists (1 \leq i \leq |S'| - k) \; S'[i : i + k) = q\} \tag{4.1}$$

Let $\mathcal{M}_{p^{l-1},m^l}$ be the set of distinct minimizers of lengths m^l on level l of the k-mers in $\mathcal{K}_{p^{l-1}}$. The set of distinct minimizers is not only affected by the choice of level parameters on previous levels but also depends on the minimizer length on the current level:

$$\mathcal{M}_{p^{l-1},m^l} := \{min_{m^l}^{h_l}(q) \mid \forall q \in \mathcal{K}_{p^{l-1}}\} \tag{4.2}$$

Let $M[p^{l-1}, m]$ be a discrete random variable that describes the frequency of these minimizers:

$$M[p^{l-1}, m] : \mathcal{M}_{p^{l-1},m^l} \to \mathbb{N} \tag{4.3}$$

$$M[p^{l-1}, m](x) = freq(x) \tag{4.4}$$

We define its probability distribution $p_{M[p^{l-1},m^l]}$[1], which tells the probability of a minimizer having an exact frequency $f \in \mathbb{N}$. The distribution of minimizers depends both on

[1]This notation differs slightly from that used by Schulte-Mattler and Reinert [SMR26] who do not introduce the probability distributions but instead use p_M and p_K to denote the cumulative distribution functions which we denote with F_M and F_K

the level parameters p^{l-1} on previous levels, which affect the remaining k-mers on level l and the minimizer length m^l on the current level, which directly affects the computation of minimizers. We thus index the distributions with those parameters:

$$p_{M[p^{l-1},m^l]} : \mathbb{N} \to [0,1] \tag{4.5}$$

$$p_{M[p^{l-1},m^l]}(f) = \mathbb{P}(M[p^{l-1},m^l] = f) = \frac{|\{x \in \mathcal{M}_{p^{l-1},m^l} \mid M[p^{l-1},m^l](x) = f\}|}{|\mathcal{M}_{p^{l-1},m^l}|} \tag{4.6}$$

Using the random variables probability function we can derive its cumulative distribution function $F_{M[p^{l-1},m]}$, which returns the probability of a minimizer having a frequency that does not exceed an upper bound f:

$$F_{M[p^{l-1},m^l]} : \mathbb{N} \to [0,1] \tag{4.7}$$

$$F_{M[p^{l-1},m^l]}(f) = \mathbb{P}(M[p^{l-1},m^l] \leq f) = \sum_{i=1}^{f} p_{M[p^{l-1},m^l]}(i) \tag{4.8}$$

Whilst $M[p^{l-1},m^l]$ and its distribution are useful for modeling values depending of a random (appearing) minimizer, RSHash is used to lookup k-mers, not minimizers. Therefore, we introduce the discrete random variable $K[p^{l-1},m^l]$ which is defined for every distinct (appearing) k-mer on the level and describes the frequency each k-mers minimizer dependent of the level parameters p^{l-1} on all previous levels and the minimizer length m^l on the current one:

$$K[p^{l-1},m^l] : \mathcal{K}_{p^{l-1}} \to \mathbb{N} \tag{4.9}$$

$$K[p^{l-1},m^l](q) = freq(min^{h_l}_{m^l}(q)) \tag{4.10}$$

Its probability distribution $p_{K[p^{l-1},m^l]}$ and its cumulative distribution function $F_{K[p^{l-1},m^l]}$ are:

$$p_{K[p^{l-1},m^l]} : \mathbb{N} \to [0,1] \tag{4.11}$$

$$p_{K[p^{l-1},m^l]}(f) = \mathbb{P}(K[p^{l-1},m^l] = f) = \frac{|\{q \in \mathcal{K}_{p^{l-1}} \mid K[p^{l-1},m^l](q) = f\}|}{|\mathcal{K}_{p^{l-1}}|} \tag{4.12}$$

$$F_{K[p^{l-1},m^l]} : \mathbb{N} \to [0,1] \tag{4.13}$$

$$F_{K[p^{l-1},m^l]}(f) = \mathbb{P}(K[p^{l-1},m^l] \leq f) = \sum_{i=1}^{f} p_{K[p^{l-1},m^l]}(i) \tag{4.14}$$

Let p^l be a combination of level parameters on a level l. Recall that we use $p^l[m^l]$ to denote retrieving the minimizer length specified for level l from the level parameters and analogously for $p^l[t^l]$. For better readability we introduce a more compact notation to denote the probability $F_{M[p^l]}$ of a minimizer being *infrequent* on level l and $F_{K[p^l]}$ of a k-mer having an *infrequent* minimizer on that level, both in dependence of all level parameters:

$$F_{M[p^l]} = F_{M[p^{l-1},p^l[m^l]]}(p^l[t^l]) \tag{4.15}$$

$$F_{K[p^l]} = F_{K[p^{l-1},p^l[m^l]]}(p^l[t^l]) \tag{4.16}$$

4.2 Average Frequencies

Let $\Omega_{M[p^{l-1},m^l]}$ be the sample space of the random variable $M[p^{l-1}, m^l]$ i.e. the set which contains all frequencies f for which there is at least one minimizer x on level l which occurs exactly f times:

$$\Omega_{M[p^{l-1},m^l]} = \{f \mid \exists x \in \mathcal{M}_{p^{l-1},m^l} freq(x) = f\} \tag{4.17}$$

The average frequency of minimizers on a level $l \in \mathcal{L}$ is given by the expected value of the probability distribution of the random variable $M[p^{l-1}, m]$:

$$\mathbb{E}[M[p^{l-1}, m^l]] = \sum_{f \in \Omega_{M[p^{l-1},m^l]}} p_{M[p^{l-1},m^l]}(f) * f \tag{4.18}$$

For a given set of finite sequences the set of distinct minimizers on each level and for each combination of level parameters is finite by definition. It follows that for a given sequence and set of level parameters, the average frequency $\mathbb{E}[M[p^{l-1}, m^l]]$ can always be computed, because the sample space $\Omega_{M[p^{l-1},m^l]}$ is then finite by definition.

More relevant to the optimization of level parameters is the average frequency of *infrequent* minimizers i.e. the average frequency of minimizers whose frequency do not exceed the threshold t^l. We denote this value with $f_{avg}(p^l)$ and can derive it from $p_{M[p^{l-1},m]}$ by computing a weighted sum over all frequencies up until the threshold t^l and then norming the result by dividing through the percentage of *infrequent* minimizers:

$$f_{avg}(p^l) = \frac{1}{F_{M[p^l]}} \cdot \sum_{f=1}^{t^l} p_{M[p^{l-1},p^l[m^l]]}(f) \cdot f \tag{4.19}$$

4.3 Numbers of Distinct Infrequent Minimizers

We define the number of distinct *infrequent* minimizers $n_{DIM}(p^l)$ which also depend on all level parameters p^l of a level l. Their value can be computed py multiplying the total number of distinct minimizers on a level with the percentage of *infrequent* ones:

$$n_{DIM}(p^l) = |\mathcal{M}_{p^{l-1},m^l}| \cdot F_{M[p^l]} \tag{4.20}$$

4.4 Times for Operations on Data Structures

-¿ Experimente??

We denote the time needed to perform an operation *op* on a data structure *struct* with $T_{op_{struct}}$ e.g. the time for a select query on the bitvector S is T_{sel_S}.

incomplete. measure or model?

Vector Access Accessing an element of a Vector can be done in constant time, we use the following values based on measurements:

$$T_{contains_R} = ...$$
$$T_{contains_O} = ...$$

Hash Set Access Accessing an element of a hash set can be done in constant time, we use the following values based on measurements:

$$T_{check_H} = ...$$

Bit Vector Rank Query A bitvector of length U with n $1s$ can be encoded with $n\lceil log_2 U/n \rceil + 3n$ many bits supporting rank in $O(log_2(U/n)$ time [OS07].

does that help here? Doesn't the model require actual time instead of O notation - maybe measurements would be more suited

Bit Vector Select Query ???
Comparison with Text ???

5 MINLP Modeling

We model the optimization problem as a Mixed Integer Non Linear Programming (MINLP). In Section 5.2 we model the lookup time and space needed by RSHash on one level in dependence of the level parameters and minimizers distribution. We then extend this to the case of several levels in Section 5.3.
In Chapter 6 we describe how to find the optimal level parameters for a given sequence using the MINLP.

5.1 Model Input

The set of levels $\mathcal{L}$, the sets of minimizer lengths $\mathcal{M} \subseteq [1, k) \cap \mathbb{N}$ and thresholds $\mathcal{T} \subset \mathbb{N}$ to consider have to be specified explicitly.
Furthermore, the number n_{kmers} of k-mers in the sequences the values $F_{M[p^l]}$, $F_{K[p^l]}$, $f_{avg}(p^l)$ and $n_{DIM}(p^l)$ for all possible combinations p^l of the level parameters on each level $l \in \mathcal{L}$ as defined in Chapter 4 have to be passed as input to the MINLP. Their computation is further explained in Chapter 6.
Lastly, a measure of the times $T_{contains_R}$, T_{sel_S}, $T_{contains_O}$, T_{comp_T}, T_{check_H} needed for the contains query on bitvector R, the select queries on bitvector S, a comparison with the text T at a random offset and checking whether a key exists in the hash set H. The computation of the minimizer distribution as well as the times needed to perform these actions on the specified data structures is outlined in Chapter 6.

5.2 Model for One Level

Aiming to improve readability, we omit the superscript index indicating an instances belonging to a specific level as only one level is considered in this section e.g. the minimizer length is simply referred to as m instead of m^1.
In order to determine $\mathbf{m}$ and $\mathbf{t}$ we use binary indicator variables μ_m and τ_t for each possible $m \in \mathcal{M}$ and $t \in \mathcal{T}$:

$$\mu_m \in \{0, 1\} \mid \forall m \in \mathcal{M} \tag{5.1}$$
$$\tau_t \in \{0, 1\} \mid \forall t \in \mathcal{T} \tag{5.2}$$

The indicator variables for each minimizer length should be mutually exclusive as $\mathbf{m}$ can only take one distinct value. The same holds for the indicator variables for each

threshold:

$$\sum_{m \in \mathcal{M}} \mu_m = 1 \tag{5.3}$$

$$\sum_{t \in \mathcal{T}} \tau_t = 1 \tag{5.4}$$

The variables $\mathbf{m} \in \mathcal{M}$ and $\mathbf{t} \in \mathcal{T}$ are introduced to describe the optimal minimizer length and threshold.
There values can be modeled through a sum over the indicator variables multiplied with there corresponding values:

$$\mathbf{m} \in \mathcal{M} \tag{5.5}$$

$$\mathbf{t} \in \mathcal{T} \tag{5.6}$$

$$\mathbf{m} = \sum_{m \in \mathcal{M}} \mu_m \cdot m \tag{5.7}$$

$$\mathbf{t} = \sum_{t \in \mathcal{T}} \tau_t \cdot t \tag{5.8}$$

We introduce the variable $\mathbf{F}_M \in [0, 1]$ to take the value of $F_{M[(\mathbf{m},\mathbf{t})]}$ i.e. the percentage of minimizers of length $\mathbf{m}$ in the input sequence, whose frequencies do not exceed $\mathbf{t}$ (Section 4.1).
It is not possible to directly evaluate this value as both $\mathbf{m}$ and $\mathbf{t}$ are variables of the model. Instead we use the indicator variables to model a constraint forcing $\mathbf{F}_M$ to take the value of $F_{M[(m,t)]}$ for which $\mu_m = 1$ and $\tau_t = 1$:

$$\mathbf{F}_M \in [0, 1] \tag{5.9}$$

$$\mathbf{F}_M = \sum_{(m,t) \in \mathcal{M} \times \mathcal{T}} F_{M[(m,t)]} \cdot \mu_m \tau_t \tag{5.10}$$

Analogously we introduce $\mathbf{F}_K \in [0, 1]$ to represent $F_{K[(\mathbf{m},\mathbf{t})]}$ and $\mathbf{f_{avg}} \in \mathbb{R}_0^+$ to represent $f_{avg}((\mathbf{m}, \mathbf{t}))$ (Section 4.2):

$$\mathbf{F}_K \in [0, 1] \tag{5.11}$$

$$\mathbf{f_{avg}} \in \mathbb{R}_0^+ \tag{5.12}$$

$$\mathbf{F}_K = \sum_{(m,t) \in \mathcal{M} \times \mathcal{T}} F_{K[(m,t)]} \cdot \mu_m \tau_t \tag{5.13}$$

$$\mathbf{f_{avg}} = \sum_{(m,t) \in \mathcal{M} \times \mathcal{T}} f_{avg}((m, t)) \cdot \mu_m \tau_t \tag{5.14}$$

5.2.1 Lookup Time

In order to model the time needed for a lookup using RSHash, we closely follow the lookup procedure as described by Schulte-Mattler and Reinert [SMR26]:

A k-mer q is looked up by first computing its minimizer $min_m^h(q)$ and checking whether the bit $R[min_m^h(q)]$ is set whilst simultaneously computing its rank $r = rank_R(min_m^h(q))$, taking $T_{contains_R}$ time (part of the model input Section 5.1).
For the percentage $\mathbf{F}_K$ (Equation (5.13)) of the k-mers this bit is set and the lookup is completed by searching for q on the level. We introduce the variable $T_{level} \in \mathbb{R}_0^+$ to describe the time needed for this.
If the bit $R[min_m^h(q)]$ is not set, the lookup is completed through a lookup in the hash set H taking T_{check_H} time (part of the model input Section 5.1). The variable $T_{lookup} \in \mathbb{R}_0^+$ describing the lookup time can thus be modeled as:

$$T_{level} \in \mathbb{R}_0^+ \tag{5.15}$$
$$T_{lookup} \in \mathbb{R}_0^+ \tag{5.16}$$
$$T_{lookup} = T_{contains_R} + \mathbf{F}_K \cdot T_{level} + (1 - \mathbf{F}_K) \cdot T_{check_H} \tag{5.17}$$

The search for those k-mers q for which the bit $R[min_m^h(q)]$ is set continues on the level by using the minimizers rank $r = rank_R(min_m^h(q))$ in the select queries $first = sel_S(r)$ and $last = sel_S(r+1) - 1$ which tell the indices in Offsets O where the minimizer's first and last offset in the text T are stored. These two queries take $2T_{sel_S}$ time.
Finally the presence of q in T is checked at every minimizer offset in the interval $O[first : last]$ until q is either found or the end of the interval is reached. The average frequency $\mathbf{f_{avg}}$ of *infrequent* minimizers is exactly the average length of this interval. In the lookup case, the comparison is terminated as soon as q is found for the first time, which is on average the case after $(\mathbf{f_{avg}}+1)/2$ comparisons with each comparison taking T_{comp_T} time.
The following constraint models the complete time T_{level} needed to search for a k-mers presence on the level:

$$T_{level} = 2T_{sel_S} + (\mathbf{f_{avg}} + 1)/2 \cdot (T_{contains_O} + T_{comp_T}) \tag{5.18}$$

Note that the value T_{lookup} is the average time for a positive RSHash lookup, that is the time to lookup a k-mer which was randomly sampled from the sequence. In practice, queried k-mers may or may not appear in the sequence - hence the need for a lookup. Looking up a k-mer which does not appear in the sequence (negative lookup) would always take $T_{contains_R} + T_{check_H}$; however, we do not account for this as the ratio of positive and negative queries is impossible predict.

Without further constraints, solving the described MINLP for minimal T_{lookup} results in a solution where the μ_m, τ_t corresponding to the smallest $m \in \mathcal{M}$, $t \in \mathcal{T}$ will be set to one.
This is expected, as this leads to the smallest possible $\mathbf{F}_K$ which minimizes the k-mers having to be looked up in the text and maximizes the lookups in the (faster) hash set.
However, this results in maximal space consumption, as each entry in the hash set is significantly larger than an entry in O as described in **??**.
In order to account for that, we have to add constraints modeling the space consumption.

explain select and rank

this sould be true for the unitigs, however, for unprocessed genomes, many offsets can belong to the same k-mer - does that change something?

5.2.2 Space Consumption

Instead of modeling the total space used by RSHash we will focus on the bits per k-mer (bpk) used by the data structure as this metric allows to compare compression independent of the sequence length.
The space consumption of RSHash can be decomposed into the space needed for the data structures components: the text T[1], the vectors R, S and O as well as the hash set H. We use one variable to describe the bits per k-mer needed by each of the components:

$$bpk_T \in \mathbb{R}_0^+ \tag{5.19}$$
$$bpk_R \in \mathbb{R}_0^+ \tag{5.20}$$
$$bpk_S \in \mathbb{R}_0^+ \tag{5.21}$$
$$bpk_O \in \mathbb{R}_0^+ \tag{5.22}$$
$$bpk_H \in \mathbb{R}_0^+ \tag{5.23}$$

The text T is much longer than k. Because of this the bits per k-mer for storing the text T can be set to a constant of 2 bits needed to store one base:

$$bpk_T = 2 \tag{5.24}$$

Bitvector R is a bitvector of length $4^{\mathbf{m}}$ in which each bit set to 1 marks the occurrence of one distinct *infrequent* minimizer in the sequences. The amount of ones in R can thus be computed by selecting the the With $n_{DIM}(p^{(m,t)})$ being the number of distinct *infrequent* minimizers in dependence of the specified level parameters, we can derive the value $\mathbf{n_{DIM}}$ for the chosen minimizer length $\mathbf{m}$ and threshold $\mathbf{t}$:

$$\mathbf{n_{DIM}} \in \mathbb{R}_0^+ \tag{5.25}$$
$$\mathbf{n_{DIM}} = \sum_{(m,t)\in\mathcal{M}\times\mathcal{T}} n_{DIM}((m,t)) \cdot \mu_m \tau_t \tag{5.26}$$

We note that for continuous sequences one could also model $\mathbf{n_{DIM}}$ by computing the number $\mathbf{F}_K \cdot n_{kmers}$ of k-mers with *infrequent* minimizers and dividing it by the average frequency $\mathbf{f_{avg}}$ of minimizers multiplied with the ratio $(k - \mathbf{m} + 2)/2$ of k-mers covered per minimizer. This should decrease the models complexity as it would replace the non-linear constraint *Equation* (5.26) with a linear one. However, the ratio $(k - \mathbf{m} + 2)/2$ is only valid for continuous sequences S of lengths $|S| \gg k > \mathbf{m}$, rendering it useless both for sequences which have been preprocessed to remove duplicate k-mers and for Section 5.3 where the input on higher levels which consists of many relatively short super-k-mers.

Using Elias-Fano-Encoding, a bitvector of size U with n bits set to one can be compressed to $n(\lceil log_2(U/n) \rceil + 3)$ [OS07] . Therefore, we can model the bits per k-mer needed

woher kommt diese formel, kann sie so im paper nicht finden

[1] Recall that we use the 'text' T to refer to the set of input sequences S_{p^0} in order to avoid confusion with the bit vector S.

to store R by using this compression and dividing this by the total number n_{kmers} of k-mers yields the constraint describing the bits per k-mer of R:

$$bpk_R = \frac{\mathbf{n_{DIM}}}{n_{kmers}} \cdot (\lceil 2\mathbf{m} - log_2\mathbf{n_{DIM}} \rceil + 3) \tag{5.27}$$

Bitvector S has length $|O|+1 \approx |O|$ where every bit corresponds to the offset of one minimizer encountered in the sequences and one extra bit marks the end of all offsets - this last bit can be disregarded when modeling bpk_S as its effect is insignificantly small. Multiplying the number $\mathbf{n_{DIM}}$ of distinct *infrequent* minimizers with their average frequency $\mathbf{f_{avg}}$ gives the length of the offset vector O. To model the bits per k-mer in S we divide this by the number of k-mers in the sequence:

$$bpk_S = \frac{\mathbf{n_{DIM}} \cdot \mathbf{f_{avg}}}{n_{kmers}} \tag{5.28}$$

Note that if one figured out, how to model the ratio between minimizers and k-mers on non-continuous sequences, bpk_S could be modeled without the use of $\mathbf{n_{DIM}}$ by taking the percentage $\mathbf{F}_K$ of k-mers with infrequent minimizers and dividing it by that ratio.

The offset vector O contains one entry of size $log_2|T|$ for the offset of each *infrequent* minimizer.
Since bpk_S already describes the number of offsets per k-mer, we can model the bits per k-mer in O with:

$$bpk_O = bpk_S \cdot log_2|T| \tag{5.29}$$

The hash set H contains an entry $(key : q)$ for each distinct k-mer q with an *infrequent* minimizer, which are $1-\mathbf{F}_K$ of all k-mers in T. Each entry has size $size(key) = 2k$ bit. We introduce the load factor[2] $\alpha = 0.75$. The bits per k-mer required for the hash set can thus be modeled by:

$$bpk_H = (1 - \mathbf{F}_K) \cdot \frac{2k}{\alpha} \tag{5.30}$$

The total bits per k-mer $bpk_{total} \in \mathbb{R}_0^+$ can be modeled by simply adding up the space requirements of each component:

$$bpk_{total} \in \mathbb{R}_0^+ \tag{5.31}$$
$$bpk_{total} = bpk_T + bpk_R + bpk_S + bpk_O + bpk_H \tag{5.32}$$

5.2.3 Optimization Goal

Having defined all constraints regarding the lookup time and space consumption of RSHash, we can now construct a MINLP in order to find optimal level parameters.

[2]In practice, the load factor of a hash set is usually not known before building the set but rather a value used for re scaling. Therefore $\alpha = 0.75$ is rather an estimate than the actual load factor of a possible hash set. We further asses the consequences of this in Chapter 6.

We introduce three more external parameters to the model: The optimization goal $OptGoal$ which specifies whether to minimize T_{lookup} or bpk_{total} and upper bounds $Bound_{time}, Bound_{bpk}$ for the lookup time and space consumption.

$$\begin{aligned} OptGoal &\in \{T_{lookup}, bpk_{total}\} \\ Bound_{time} &\in \mathbb{R}_0^+ \\ Bound_{bpk} &\in \mathbb{R}_0^+ \end{aligned}$$

When using the MINLP, the upper bound belonging to the optimization goal should be set to ∞. In addition to the already defined ones, the following two constraints are added to the MINLP to enforce these bounds:

$$T_{lookup} \leq Bound_{time} \tag{5.33}$$
$$bpk_{total} \leq Bound_{bpk} \tag{5.34}$$

One could for example search for level parameters minimizing the lookup time whilst not exceeding eight bits per k-mer by setting $OptGoal \leftarrow T_{lookup}, Bound_{time} \leftarrow \infty, Bound_{bpk} \leftarrow 8$. The constructed MINLP for this example is:

$$\begin{aligned} INPUT : \; & OptGoal = T_{lookup}, Bound_{time} = \infty, Bound_{bpk} = 8, \\ & \mathcal{L} = \{1\}, \mathcal{M}, \mathcal{T}, n_{kmers} \\ & F_{M[(m,t)]}, F_{K[(m,t)]}, f_{avg}((m,t)), n_{DIM}((m,t)) \; \forall (m,t) \in \mathcal{M} \times \mathcal{T}, \\ & T_{contains_R}, T_{sel_S}, T_{contains_O}, T_{comp_T}, T_{check_H} \end{aligned}$$

MINIMIZE

$$OptGoal = T_{lookup}$$

SUBJECT TO

$$\sum_{m \in \mathcal{M}} \mu_m = 1 \qquad \textit{Equation (5.3)}$$

$$\sum_{t \in \mathcal{T}} \tau_t = 1 \qquad \textit{Equation (5.4)}$$

$$\mathbf{m} = \sum_{m \in \mathcal{M}} \mu_m \cdot m \qquad \textit{Equation (5.7)}$$

$$\mathbf{t} = \sum_{t \in \mathcal{T}} \tau_t \cdot t \qquad \textit{Equation (5.8)}$$

$$\mathbf{F}_M = \sum_{(m,t) \in \mathcal{M} \times \mathcal{T}} F_{M[(m,t)]} \cdot \mu_m \tau_t \qquad \textit{Equation (5.10)}$$

$$\mathbf{F}_K = \sum_{(m,t) \in \mathcal{M} \times \mathcal{T}} F_{K[(m,t)]} \cdot \mu_m \tau_t \qquad \textit{Equation (5.13)}$$

$$\mathbf{f_{avg}} = \sum_{(m,t) \in \mathcal{M} \times \mathcal{T}} f_{avg}((m,t)) \cdot \mu_m \tau_t \qquad \textit{Equation (5.14)}$$

$$T_{level} = 2T_{sel_S} + (\mathbf{f_{avg}} + 1)/2 \cdot (T_{contains_O} + T_{comp_T}) \qquad \textit{Equation (5.18)}$$

$$T_{lookup} = T_{contains_R} + \mathbf{F}_K \cdot T_{level} + (1 - \mathbf{F}_K) \cdot T_{check_H} \qquad \textit{Equation (5.17)}$$

$$T_{lookup} \leq Bound_{time} = \infty \qquad \textit{Equation (5.33)}$$

$$bpk_T = 2 \qquad \textit{Equation (5.24)}$$

$$\mathbf{n_{DIM}} = \sum_{(m,t) \in \mathcal{M} \times \mathcal{T}} n_{DIM}((m,t)) \cdot \mu_m \tau_t \qquad \textit{Equation (5.26)}$$

$$bpk_R = \frac{\mathbf{n_{DIM}}}{n_{kmers}} \cdot (\lceil 2\mathbf{m} - log_2 \mathbf{n_{DIM}} \rceil + 3) \qquad \textit{Equation (5.27)}$$

$$bpk_S = \frac{\mathbf{n_{DIM}} \cdot \mathbf{f_{avg}}}{n_{kmers}} \qquad \textit{Equation (5.28)}$$

$$bpk_O = bpk_S \cdot log_2 |T| \qquad \textit{Equation (5.29)}$$

$$bpk_H = (1 - \mathbf{F}_K) \cdot \frac{2k}{\alpha} \qquad \textit{Equation (5.30)}$$

$$bpk_{total} = bpk_T + bpk_R + bpk_S + bpk_O + bpk_H \qquad \textit{Equation (5.32)}$$

$$bpk_{total} \leq Bound_{bpk} = 8 \qquad \textit{Equation (5.34)}$$

VARIABLES

$\mu_m \in \{0,1\} \mid \forall m \in M$	*Equation* (5.1)
$\tau_t \in \{0,1\} \mid \forall t \in T$	*Equation* (5.2)
$\mathbf{m} \in \mathcal{M}$	*Equation* (5.5)
$\mathbf{t} \in \mathcal{T}$	*Equation* (5.6)
$\mathbf{F}_M \in [0,1]$	*Equation* (5.9)
$\mathbf{F}_K \in [0,1]$	*Equation* (5.11)
$\mathbf{f_{avg}} \in \mathbb{R}_0^+$	*Equation* (5.12)
$T_{level} \in \mathbb{R}_0^+$	*Equation* (5.15)
$T_{lookup} \in \mathbb{R}_0^+$	*Equation* (5.16)
$bpk_T \in \mathbb{R}_0^+$	*Equation* (5.19)
$bpk_R \in \mathbb{R}_0^+$	*Equation* (5.20)
$bpk_S \in \mathbb{R}_0^+$	*Equation* (5.21)
$bpk_O \in \mathbb{R}_0^+$	*Equation* (5.22)
$bpk_H \in \mathbb{R}_0^+$	*Equation* (5.23)
$\mathbf{n_{DIM}} \in \mathbb{R}_0^+$	*Equation* (5.25)
$bpk_{total} \in \mathbb{R}_0^+$	*Equation* (5.31)

5.3 Model for Several Levels

In the following, we adapt the constraints defined for using only one level in Section 5.2 to model the use of $L \in \mathbb{N}^+$ levels with $\mathcal{L} = \{l \in \mathbb{N}^+ \mid l \leq L\}$ being the set of levels used[3].

When modeling several levels, we add a superscript l to variables in order to indicate there belonging to a level $l \in \mathcal{L}$ e.g. $\mathbf{m}^1, \mathbf{m}^2$ and $\mathbf{m}^3$ denote the optimal minimizer lengths on level $1, 2$ and 3.

We still assume, that the times $T_{contains_{R^l}}, T_{sel_{S^l}}, T_{contains_{O^l}}, T_{comp_T}, T_{check_H}$ are (practically) independent of the parameters $\mathbf{m}^l, \mathbf{t}^l$ and the level l.

reason in chap: experiments

Unique sets of binary indicator variables μ_m^l and τ^l are needed for the level parameters $\mathbf{m}^l$ and $\mathbf{t}^l$ of each level l. The constraints regarding these variables are analog to those

[3] We model the space consumption and lookup time for any positive natural number of levels even though RSHash currently only supports the use of one, two or three levels [SMR26].

when using only one level:

$$\begin{aligned}
\mu_m^l &\in \{0,1\} && \mid \forall (l,m) \in (\mathcal{L} \times \mathcal{M}) \\
\tau_t^l &\in \{0,1\} && \mid \forall (l,t) \in (\mathcal{L} \times \mathcal{T}) \\
\sum_{m \in \mathcal{M}} \mu_m^l &= 1 && \mid \forall l \in \mathcal{L} \\
\sum_{t \in \mathcal{T}} \tau_t^l &= 1 && \mid \forall l \in \mathcal{L} \\
\mathbf{m}^l &\in \mathbb{R}_0^+ && \mid \forall l \in \mathcal{L} \\
\mathbf{t}^l &\in \mathbb{R}_0^+ && \mid \forall l \in \mathcal{L} \\
\mathbf{m}^l &= \sum_{m \in \mathcal{M}} \mu_m^l \cdot m^l && \mid \forall l \in \mathcal{L} \\
\mathbf{t}^l &= \sum_{t \in \mathcal{T}} \tau_t^l \cdot t^l && \mid \forall l \in \mathcal{L}
\end{aligned}$$

For each level l we model $\mathbf{F}_M^l$, $\mathbf{F}_K^l$ and $\mathbf{f}_{\mathbf{avg}}^l$ analogously to using a single level; however, it is necessary to sum over every possible combination of level parameters p^l. The number of additions thus grows exponentially for an increasing number of levels.

$$\begin{aligned}
&\mathbf{F}_M^l \in [0,1] && \mid \forall l \in \mathcal{L} \\
&\mathbf{F}_K^l \in [0,1] && \mid \forall l \in \mathcal{L} \\
&\mathbf{f}_{\mathbf{avg}}^l \in \mathbb{R}_0^+ && \mid \forall l \in \mathcal{L} \\
&\mathbf{F}_M^l = \sum_{p^l \in \mathcal{M}^l \times \mathcal{T}^l} F_{M[p^l]} \cdot \prod_{1 \le i \le l} \mu^i_{p^l[m^i]} \tau^i_{p^l[t^i]} && \mid \forall l \in \mathcal{L} \\
&\mathbf{F}_K^l = \sum_{p^l \in \mathcal{M}^l \times \mathcal{T}^l} F_{K[p^l]} \cdot \prod_{1 \le i \le l} \mu^i_{p^l[m^i]} \tau^i_{p^l[t^i]} && \mid \forall l \in \mathcal{L} \\
&\mathbf{f}_{\mathbf{avg}}^l = \sum_{p^l \in \mathcal{M}^l \times \mathcal{T}^l} f_{avg}(p^l) \cdot \prod_{1 \le i \le l} \mu^i_{p^l[m^i]} \tau^i_{p^l[t^i]} && \mid \forall l \in \mathcal{L}
\end{aligned}$$

5.3.1 Lookup Time

The search time on a level $l \in \mathcal{L}$ can be leveled analogously to that on one level:

$$\begin{aligned}
&T_{level}^l \in \mathbb{R}_0^+ && \mid \forall l \in \mathcal{L} \\
&T_{level}^l = 2T_{sel_{S^2}} + \frac{\mathbf{f}_{\mathbf{avg}}^l + 1}{2} \cdot (T_{contains_{O^2}} + T_{comp_T}) && \mid \forall l \in \mathcal{L}
\end{aligned}$$

For $\mathbf{F}_K^l$ of the k-mers on a level $l \in \mathcal{L}$ the lookup is completed by a search through the offsets on that level whilst the remaining $(1 - \mathbf{F}_K^l)$ of queries result in a lookup on the next level.
We can thus model the lookup time recursively:

$$T_{lookup}^l = \begin{cases} T_{contains_{R^l}} + \mathbf{F}_K^l \cdot T_{level}^l \\ \qquad\qquad + (1 - \mathbf{F}_K^l) \cdot T_{lookup}^{l+1} & \mid l \in \mathcal{L} \\ T_{check_H} & \mid l \notin \mathcal{L} \end{cases}$$

Considering the set $\mathcal{L} = \{1, 2\}$ of two levels as an example, this results in the following total lookup time:

$$\begin{aligned} T^1_{lookup} = T_{contains_{R^1}} &+ \mathbf{F}^1_K \cdot T^1_{level} \\ &+ (1 - \mathbf{F}^1_K) \cdot \Big(T_{contains_{R^2}} + \mathbf{F}^2_K \cdot T^2_{level} \\ &\qquad + (1 - \mathbf{F}^2_K) \cdot T_{check_H} \Big) \end{aligned}$$

Inserting T^1_{level} and T^2_{level} yields:

$$\begin{aligned} T^1_{lookup} = T_{contains_{R^1}} &+ \mathbf{F}^1_K \cdot (2T_{sel_{S^1}} + \frac{\mathbf{f}^1_{\mathbf{avg}} + 1}{2} \cdot (T_{contains_{O^1}} + T_{comp_T})) \\ &+ (1 - \mathbf{F}^1_K) \cdot \Bigg(T_{contains_{R^2}} + \mathbf{F}^2_K \cdot (2T_{sel_{S^2}} + \frac{\mathbf{f}^2_{\mathbf{avg}} + 1}{2} \cdot (T_{contains_{O^2}} + T_{comp_T})) \\ &\qquad + (1 - \mathbf{F}^2_K) \cdot T_{check_H} \Bigg) \end{aligned}$$

insertion weglassen?

5.3.2 Space Consumption

The text T is only being stored once, so the constant approximation $bpk_T \approx 2$ can be used for several levels as well.

The number of distinct *infrequent* minimizers $\mathbf{n}^l_{\mathbf{DIM}}$ on each level l can modeled analogously to $\mathbf{F}^l_M$, $\mathbf{F}^l_K$ and $\mathbf{f}^l_{\mathbf{avg}}$:

$$\begin{aligned} &\mathbf{n}^l_{\mathbf{DIM}} \in \mathbb{R}^+_0 && \mid \forall l \in \mathcal{L} \\ &\mathbf{n}^l_{\mathbf{DIM}} = \sum_{p^l \in \mathcal{M}^l \times \mathcal{T}^l} n_{DIM}(p^l) \cdot \prod_{1 \le i \le l} \mu^i_{p^l[m^i]} \tau^i_{p^l[t^i]} && \mid \forall l \in \mathcal{L} \end{aligned}$$

Vectors R^l, S^l and O^l: The bits per k-mer needed to store the bit vectors R^l and S^l as well as the offset vector O^l on every level l can be modeled analogously to using only one level:

$$\begin{aligned} &bpk_{R^l} \in \mathbb{R}^+_0 && \mid \forall l \in \mathcal{L} \\ &bpk_{S^l} \in \mathbb{R}^+_0 && \mid \forall l \in \mathcal{L} \\ &bpk_{O^l} \in \mathbb{R}^+_0 && \mid \forall l \in \mathcal{L} \\ &bpk^l_R = \frac{\mathbf{n}^l_{\mathbf{DIM}}}{n_{kmers}} \cdot \left(\lceil 2\mathbf{m}^l - log_2 \mathbf{n}^l_{\mathbf{DIM}} \rceil + 3 \right) && \mid \forall l \in \mathcal{L} \\ &bpk_{S^l} = \frac{\mathbf{n}^l_{\mathbf{DIM}} \cdot \mathbf{f}^l_{\mathbf{avg}}}{n_{kmers}} && \mid \forall l \in \mathcal{L} \\ &bpk_{O^l} = bpk_{S^l} \cdot log_2 |T| && \mid \forall l \in \mathcal{L} \end{aligned}$$

The hash set H only contains an entry for k-mers q, which have frequent minimizers on all levels. The bits per k-mer for the hash set can thus be modeled as

$$bpk_H = \left(\prod_{l=1}^{L} 1 - \mathbf{F}_K^l \right) \cdot \frac{64}{\alpha}$$

The total bits per k-mer for L levels only differ to those on one level, as there are now a vector R, S and O on each level:

$$bpk_{total} = bpk_T + \sum_{l=1}^{L} (bpk_{R^l} + bpk_{S^l} + bpk_{O^l}) + bpk_H$$

6 Experiments

We evaluate how suited the MINLP proposed in Chapter 5 is for finding optimal level parameters by analyzing how accurate the modeled lookup time and space consumption are compared to the actual lookup time measured and space needed when building RSHash. We then examine how well the model can be optimized using the SCIP solver [Ach+08; Ach09; Bes+23]. Whether the resulting parameters are actually optimal in practice is highly dependent on how accurate the model is in the first place.

6.1 Datasets

We use the same datasets as Pibiri [Pib22]. Those are the whole genomes of the atlantic cod (*Gadus morhua*), the common kestrel (*Falco tinnunculus*) and the whole GRCh38 human genome (*Homo sapiens*) This choice allows a direct comparison with Schulte-Mattler and Reinert [SMR26] who use the same datasets for benchmarking RSHash.
We assess the proposed MINLP both on the datasets themselves as well as on preprocessed versions without duplicate k-mers. For this a compacted de Bruijn Graph is built with BCALM2 [CLM16] whose maximal unitigs (non-branching paths) are extracted using the tool UST [RM21].

6.2 Considered Parameters

For the following experiments we use k-mers of length $k = 31$ and the largest number of levels $L = 3$ that is currently supported by RSHash.
RSHash currently works with thresholds up to 255. Technically, using the proposed MINLP to find optimal level parameters for a specified optimization goal, one would have to consider every minimizer lengths $1 \leq m < k$ and all thresholds $1 \leq t \leq 255$. However, using the proposed model requires computing the distributions of minimizers for each possible combination of parameters. In practice using 3 levels would require computing the distributions for $30^3 \cdot 255^3 \approx 448 \cdot 10^9$ combinations of level parameters which is is not viable.

When using the maximum threshold $t = 255$ and minimizers of length 15 leads to a percentage of *infrequent* minimizers below *percentage* for each of the datasets considered. As this already leads to a comparatively large hash set, minimizer lengths $m < 15$ can be disregarded when searching for level parameters which lead to relatively low query

percentage einfügen!!

times whilst also keeping the data structure fairly compact.
When using a small threshold $t = 32$ the percentage of *infrequent* minimizers of length 25 is above *percentage* for each of the datasets considered. Whilst this would lead to a small hash set, the compression would be small, as each minimizer covers only approximately $(31 - 25 + 2)/2 = 4$ k-mers. Because of this, considering minimizer lengths between fifteen and twenty five should be sufficient for analyzing the model. To further reduce the number of parameter combinations we only consider uneven minimizer lengths.

$$\mathcal{M} = \{m \in \mathbb{N} \mid 15 \leq m \leq 25 \wedge m\%2 = 1\} \tag{6.1}$$

Recall that cumulative distribution functions $F_{M[p^l]}$ and $F_{K[p^l]}$ have logarithmic growth. It follows that using exponentially increasing thresholds should be sufficient for analyzing the MINLP. We consider the thresholds $32, 64$ and 128.

$$\mathcal{T} = \{32, 64, 128\} \tag{6.2}$$

With these sets of considered parameters, using 3 levels would result in $6^3 \cdot 3^3 = 5,832$ combinations of level parameters. We reduce this even further by considering only non-decreasing parameter values on subsequent levels for both the minimizer lengths and thresholds. That means each combination of level parameters $p^3 = (m^1, m^2, m^3, t^1, t^2, t^3)$ fulfills $m^i \leq m^i + 1$ and $t^i \leq t^i + 1$ for all $i \in \{1, 2\}$ which can be done because.

explain the constraint enforcing this in the MINLP and how the sum over possible minimizer combinations changes

$$constraints... \tag{6.3}$$

For a non-decreasing sequence of length k', chosen from n values, the number of possibilities is $\binom{n+k'-1}{k'}$. That means that when choosing three minimizer lengths out of six possible values and three thresholds out of three possible values, the total number of possible parameter combinations is $\binom{6+3-1}{3} \cdot \binom{3+3-1}{3} = 560$.

6.3 Computing the Model Input

In order to evaluate the accuracy of the proposed MINLP, we first show how to compute the model input for each of the original/preprocessed datasets which we refer to as the set of sequences S_{p^0}. This is done for $L = 3$ levels and non-decreasing level parameters as described in Section 6.2.
The required stats are the total number n_{kmers} of k-mers in the input sequence and the cumulative distributions of minimizer frequencies $F_{M[p^l]}$ and $F_{K[p^l]}$, the average frequencies $f_{avg}(p^l)$ and the number of distinct *infrequent* minimizers $n_{DIM}(p^l)$ for every non-decreasing combination of level parameters $p^l \in \mathcal{M}^l \times \mathcal{T}^l$ on every level $1 \leq l \leq 3$.

These stats can be brute-forced for every dataset considered by iterating over each non-decreasing combination of the specified level parameters and actually building RSHash

with those parameters. We use the RSHash algorithm implemented by [SMR26] which is available at https://github.com/jonsmcode/rshash.
Whilst there are more efficient approaches to only compute the model input, we chose this approach as it not only computes an accurate model input but also allows to measure space consumption of the data structures components and the operation times on each of these components which we will later use to assess the accuracy of the values modeled by the MINLP.
Specifically we measure the bits per k-mer of the vectors R, S and O on each level as well as the bits per k-mer to store the text and hash set which are only stored once and not per level. We measure the operation times $T_{contains_{R^l}}$, $T_{sel_{S^l}}$, $T_{contains_{O^l}}$, T_{comp_T} and T_{check_H} needed for the contains query on bitvector R, the select queries on bitvector S, a comparison with the text T at a random offset and checking whether a key exists in the hash set H. We also measure the average lookup time T_{lookup}. These times are average times of performing each operation 100000 subsequent times.

Algorithm 1 Computation of Minimizer Distribution

Input: sequences S_{p^0}, max Level L, considered minimizer lengths and thresholds $\mathcal{M}, \mathcal{T}$
function STATSCOMPUTATION($S_{p^0}, L, \mathcal{M}, \mathcal{T}$)
$n_{kmers} \leftarrow ccountKmers(S_{p^0})$
for $p^L \in \mathcal{M}^L \times \mathcal{T}^L$ **do**
if $p^L.isDecreasing()$ **then**
continue
end if
$RSHash \leftarrow buildWithStats(S_{p^0}, p^L)$
$F_{M[p^l]}, F_{K[p^l]}, f_{avg}(p^l), n_{DIM}(p^l) \leftarrow RSHash.getDistributionStats()$
$spaceConsumptions_{p^L} \leftarrow RSHash.getSpaceConsumptions()$
$operationTimes_{p^L} \leftarrow RSHash.measureTimes(noSamples = 100000)$
end for
end function

Explain results of minimizer distribution across several levels

- higher levels distributions are highly dependent on previous parameters (hypothesis: highly repetitive regions are more likely to have *infrequent* minimizers on every level regardless of the hash functions used)
- probability functions of minimizer distributions always follow a log normal / log logistic distribution?? Because there cumulative probability function has logarithmic growth, considering exponentially growing thresholds like $2, 4, 8, 16, 32, 64, 128$ is a viable choice (the change between 2 and 4 is proportional (equal?) to that between 64 and 128)
- outlook: it might be possible to only compute a selection of distributions and extrapolate the remaining ones

6.4 Model Accuracy

We asses the accuracy of the lookup time and space consumption modeled by the MINLP for each of the datasets considered. To do this we first compute model input, the actual space consumption and the measured operation times for every non-decreasing combination of level parameters on three levels as outlined in Section 6.3.
For each original and preprocessed sequence and each combination of the level parameters considered, we then use a modified version of the proposed model which forces the exact combination of level parameters and returns the modeled space consumption and lookup time.

6.4.1 Lookup Time

6.4.2 Space Consumption

Absatz zu bpk_{total} schreiben

For both the original and preprocessed datasets the modeled space consumption of the vectors R^l, S^l and O^l on every level $1 \leq l \leq 3$ is accurate up to an error of ...
The only significant deviations are for the bits per k-mer bpk_T and bpk_H needed to store the text and the hash set H. The chosen approximation of $bpk_T \approx 2$ holds for the original sequences because those sequences are generally much longer than k. This is not necessarily the case when the input sequences have been preprocessed to remove duplicate k-mers. The preprocessed sequences are represented through unitigs, which are non-branching paths in the compact de Bruijn graph of the sequence. For highly repetitive genomes, their lengths are much closer to k, requiring more bits per k-mer to store. This effect is negligent for the common kestrels genome which contains almost no duplicate k-mers. The preprocessed genomes of atlantic cods and human need approximately 2.287 and 2.312 bits per k-mer. This could be accounted for by including the bits per k-mer needed to store a sequence in the model input as its computation is simple compared to the other stats which have to be computed for the model input.

max error eintragen

Frage: soll ich den Abschnitt bezüglich bpk_T hier in der auswertung lassen oder die bpk_T als Parameter zum Model hinzufügen (diese zu berechnen ist im Vergleich mit den restlichen stats sehr günstig)?

RSHash is implemented in c++ using $gtl :: flat_hash_set$ which uses one byte of metadata for each entry in the hash set to allow faster queries. For a direct comparison of the measured and modeled space consumption, we can account for this by modifying the constraint to be $bpk_H = \left(\prod_{l=1}^{3} 1 - \mathbf{F}_K^l\right) \cdot \frac{64+8}{\alpha}$ which is equivalent to adding a bias term of $(1 - F_{K[p^1]})(1 - F_{K[p^2]})(1 - F_{K[p^3]}) \cdot 8/0.75$ to the modeled values.
However, even with this adjustment the modeled space consumption of the hash set is neither correct for the original genomes nor for the preprocessed ones. Whilst we model the bits per k-mer bpk_H of the hash set as a continues function, our measurements seem to take values from a discrete distribution.

This observation can also be explained by the hash sets implementation: Upon declaration a c++ hash set starts of with a hash function mapping keys to a hash space of a relatively small size. That size is doubled whenever a certain criteria regarding the hash sets load is no longer fulfilled. This discrete behavior with varying loads cannot be captured by a continues model using a constant load factor.
In addition to this, the model tends to overestimate the hash sets size for the original sequences. This is likely because repeating k-mers can count towards the percentage of k-mers with *infrequent* minimizers on all levels and thus lead to a larger value of the modeled bpk_H. However, in practice these duplicate k-mers collapse to single entry in the hash set as the lookup only requires determining a single occurrence. This effect is again negligent for the common kestrels genome, where

Plots? Wenn ja, welche?

6.5 Limitations

The poorly modeled lookup time renders the MINLP useless for finding optimal level parameters for a specified optimization goal.

This approaches plausibility is heavily dependent on the time needed to compute the model input as outlined Section 6.3. Especially the runtime for the computation of the distributions $p_{M[p^{l-1},p^l[m^l]]}$ and $p_{K[p^{l-1},p^l[m^l]]}$ for every combination of the level parameters p^l on every level l grows exponentially with the number of levels when computed by building RSHash.

For this reason, an approach yielding only an approximation of the optimal level parameters but taking less time to do so could be of interest.

We will further explore methods **??** the genome in order to reduce the computation of minimizer distribution. TODO

In another approach we will try to approximate the distributions on higher levels based on that on the first level in order to get around the exponential cost of computing those values. TODO

7 Outlook

model negative lookups with a parameter for the ratio of positive/negative lookups

Locate

outline differences to lookup

- lookup time: check every offset instead of only an average of half of the offsets
- hash table instead of hash set - contains one entry (key:k-mer, value:offset) for each appearance of every k-mer (no longer distinct) - for the unitigs, this shouldn't have an effect
- Locate benötigt eine Liste mit allen offsets: $bpk_H = (1 - \mathbf{F}_K) \cdot \frac{(64+(log_2|T| \cdot frequency))}{\alpha}$

Sub sampling For large genomes, the computing of the model input could be sped up by randomly sub sampling a certain percentage of the genomes k-mers and computing the minimizer distributions on those. A smaller percentage would then mean a faster computation at the cost of losing accuracy.

8 Conclusion

9 stuff

A fairly straightforward way of computing these is to use a recursion over the levels. The input for each recursive call consist of the current level l, the number of levels L, the previous level parameters p^{l-1} and the set of input sequences in dependence of those parameters.

zu umgangssprachlich?

Upon being called, the first minimizer $m \in \mathcal{M}$ is selected and used to compute the distribution stats for the input sequence on this level in dependence of m. The function then loops over each threshold $t \in \mathcal{T}$, removes all k-mers with *infrequent* minimizers when using (m, t) from the input sequences and then calls the recursion for the next level, which receives the next level, the previous parameters extended by (m, t) and the modified sequences as input. Only once this recursion is completed for every threshold $t \in \mathcal{T}$, the next minimizer is picked from $\mathcal{M}$. The described algorithm:

soll ich hier erklären, wie die Berechnung in der Praxis noch ein bisschen schneller geschieht? Oder ist das an dieser Stelle egal, wenn es am grundlegenden Problem der exponentiellen Laufzeit nichts ändert?

Algorithm 2 Computation of Minimizer Distribution

Input: level l, max Level L, level parameters p^{l-1}, sequences $S_{p^{l-1}}$

```
function RecursiveMinimizerDistribution(l, L, p^{l-1} S_{p^{l-1}})
    if l = L then
        return
    end if
    for m ∈ M do
        for t ∈ T do
            p^l ← extend(p^{l-1}, m, t)            ▷ extend((m^1, t^1), m^2, t^2) = (m^1, m^2, t^1, t^2)
            S_{p^l} ← getFrequentKmers(l, S_{p^{l-1}}, m, t)

            F_{M[p^l]}, F_{K[p^l]}, f_avg(p^l), n_DIM(p^l) ← distributionStats(h_l, S_{p^{l-1}}, m)
            RecursiveMinimizerDistribution(l + 1, L, p^l, S_{p^l})
        end for
    end for
end function
```

This approach is guaranteed to compute correct minimizer distributions for every set of level parameters on every level as it effectively performs the same steps as the RSHash algorithm only without building the data structure itself. This also means, that one could include the building of bitvectors R, S and O on every level as well as

the hash set H and simply measure the space consumption and lookup time for every combination, which would render the MINLP obsolete.
However, this approach is sufficient for computing the input needed for analyzing how well the MINLP models the lookup time and space consumption using only the minimizer distributions. Possibilities of speeding up the time needed are outlined in Chapter 7.

Bibliography

[Ach+08] T. Achterberg, T. Berthold, T. Koch, and K. Wolter. *Constraint Integer Programming: a New Approach to Integrate CP and MIP*. eng. Tech. rep. 08-01. Takustr. 7, 14195 Berlin: ZIB, 2008 (cit. on p. 26).

[Ach09] T. Achterberg. "SCIP: solving constraint integer programs". In: *Mathematical Programming Computation* 1.1 (2009), pp. 1–41. URL: https://doi.org/10.1007/s12532-008-0001-1 (cit. on p. 26).

[Bes+23] K. Bestuzheva, A. Chmiela, B. MÃ¼ller, F. Serrano, S. Vigerske, and F. Wegscheider. *Global Optimization of Mixed-Integer Nonlinear Programs with SCIP 8*. Tech. rep. 2301.00587. arXiv, 2023 (cit. on p. 26).

[Chi+14] R. Chikhi, A. Limasset, S. Jackman, J. Simpson, and P. Medvedev. *On the representation of de Bruijn graphs*. en. arXiv:1401.5383 [q-bio.QM]. Oct. 2014. URL: http://arxiv.org/abs/1401.5383 (visited on 09/18/2026) (cit. on p. 6).

[CLM16] R. Chikhi, A. Limasset, and P. Medvedev. "Compacting de Bruijn graphs from sequencing data quickly and in low memory". en. In: *Bioinformatics* 32.12 (June 2016), pp. i201–i208. URL: https://academic.oup.com/bioinformatics/article/32/12/i201/2289008 (visited on 11/09/2026) (cit. on p. 26).

[Jai+20] C. Jain, A. Rhie, H. Zhang, C. Chu, B. P. Walenz, S. Koren, and A. M. Phillippy. "Weighted minimizer sampling improves long read mapping". en. In: *Bioinformatics* 36.Supplement_1 (July 2020), pp. i111–i118. URL: https://academic.oup.com/bioinformatics/article/36/Supplement_1/i111/5870473 (visited on 09/18/2026) (cit. on p. 6).

[Moe+24] C. Moeckel, M. Mareboina, M. A. Konnaris, C. S. Chan, I. Mouratidis, A. Montgomery, N. Chantzi, G. A. Pavlopoulos, and I. Georgakopoulos-Soares. "A survey of k-mer methods and applications in bioinformatics". en. In: *Computational and Structural Biotechnology Journal* 23 (Dec. 2024), pp. 2289–2303. URL: https://spj.science.org/doi/10.1016/j.csbj.2024.05.025 (visited on 11/02/2026) (cit. on p. 5).

[OS07] D. Okanohara and K. Sadakane. "Practical entropy-compressed rank/select dictionary". In: *2007 Proceedings of the ninth workshop on algorithm engineering and experiments (ALENEX)*. SIAM. 2007, pp. 60–70 (cit. on pp. 14, 18).

[Pib22] G. E. Pibiri. "Sparse and skew hashing of K-mers". en. In: *Bioinformatics* 38.Supplement_1 (June 2022), pp. i185–i194. URL: https://academic.oup.com/bioinformatics/article/38/Supplement_1/i185/6617506 (visited on 07/06/2026) (cit. on pp. 6–8, 26).

[RM21] A. Rahman and P. Medevedev. "Representation of k-Mer Sets Using Spectrum-Preserving String Sets". In: *Journal of Computational Biology* 28.4 (2021), pp. 381–394. eprint: https://doi.org/10.1089/cmb.2020.0431. URL: https://doi.org/10.1089/cmb.2020.0431 (cit. on p. 26).

[SMR26] J. Schulte-Mattler and K. Reinert. "RSHash: a fast and space-efficient hash table for k -mers". en. In: *Bioinformatics* 42.Supplement_2 (Aug. 2026), btag440. URL: https://academic.oup.com/bioinformatics/article/doi/10.1093/bioinformatics/btag440/8767308 (visited on 09/15/2026) (cit. on pp. 6, 7, 11, 16, 22, 26, 28).

[ZKM20] H. Zheng, C. Kingsford, and G. Marçais. "Improved design and analysis of practical minimizers". en. In: *Bioinformatics* 36.Supplement_1 (July 2020), pp. i119–i127. URL: https://academic.oup.com/bioinformatics/article/36/Supplement_1/i119/5870484 (visited on 07/06/2026) (cit. on p. 6).